

数组组成的最小数字

题目描述

给定一个整型数组，请从该数组中选择3个元素 **组成最小数** 字并输出（如果数组长度小于3，则选择数组中所有元素来组成最小数字）。

输入描述

一行用半角逗号分割的 **字符串** 记录的整型数组

- 0 < 数组长度 ≤ 100
- 0 < 整数的取值范围 ≤ 10000

输出描述

由3个元素组成的最小数字，如果 **数组长度** 小于3，则选择数组中所有元素来组成最小数字。

用例

输入	21,30,62,5,31
输出	21305
说明	数组长度超过3，需要选3个元素组成最小数字，21305由21,30,5三个元素组成的数字，为所有组合中最小的数字。

输入	5,21
输出	215
说明	数组长度小于3，选择所有元素来组成最小值，215为最小值。

题目解析

本题直接对数组进行字典序升序，拼接后得到的组合数，不一定是最小的，比如数组 [3, 32, 321]，此时按照字典序升序后，还是 [3, 32, 321]，拼接出来为 332321，而这显然不是最小的组合数，最小的组合数应该是321323。

对于三个数及以上的数组，我们需要从中取出3个数，这3个数，首先需要保证总长度最短，即保证组合数的位数最少，其值才能最小，因此我们需要将数组（按数值大小）升序，这样小数在前，大数在后，在保证总长度最小的同时，也能保证局部元素最优。比如 [1,23,456,457,2111]，取数值升序后的前三个数1, 23, 456，是最优的。

获得最优前三个元素后，按照如下规则排序：

尝试对要组合的两个数字串 a, b 进行组合，此时有两种组合方式：

- $a + b$
- $b + a$

然后，比较 $(a + b)$ 和 $(b + a)$ 的数值谁小，如果 $a + b$ 的数值更小，则保持当前 a,b 顺序不变，如果 $b + a$ 的数值更小，则交换 a,b 位置。

比如 `strs = [a, b]`, `a = "4"`, `b = "34"`

- $a + b = "434"$
- $b + a = "344"$

可以发现 $b + a$ 更小，因此交换 a,b 位置，strs变为[34, 3]，然后进行元素拼接得到 343 最大组合数。

JS算法源码

```
1  const rl = require("readline").createInterface({ input: process.stdin });
2  var iter = rl[Symbol.asyncIterator]();
3  const readline = async () => (await iter.next()).value;
4
5  void (async function () {
6      const nums = (await readline()).split(",");
7
8      nums.sort((a, b) => a - b);
9  })
```

运行

```

10 |   const ans = nums
    |         11 |   .slice(0, 3)
12 |   .sort((a, b) => {
13 |     const s1 = a + b;
14 |     const s2 = b + a;
15 |     return s1 == s2 ? 0 : s1 > s2 ? 1 : -1;
16 |   })
17 |   .join("");
18 |
19 |   console.log(ans);
20 | })();

```

Java算法源码

```

1 | import java.util.Arrays;
2 | import java.util.Scanner;
3 |
4 | public class Main {
5 |     public static void main(String[] args) {
6 |         Scanner sc = new Scanner(System.in);
7 |
8 |         String[] nums = sc.nextLine().split(",");
9 |         Arrays.sort(nums, (a, b) -> Integer.parseInt(a) - Integer.parseInt(b));
10 |
11 |         nums = Arrays.copyOfRange(nums, 0, Math.min(3, nums.length));
12 |         Arrays.sort(nums, (a, b) -> (a + b).compareTo(b + a));
13 |
14 |         for (String num : nums) {
15 |             System.out.print(num);
16 |         }
17 |     }
18 | }

```

Python算法源码

```
1 import functools
2
3 # 输入获取
4 nums = input().split(",")
5
6
7 # 算法入口
8 def cmp(a, b):
9     s1 = a + b
10    s2 = b + a
11    return 0 if s1 == s2 else 1 if s1 > s2 else -1
12
13
14 def solution():
15     nums.sort(key=lambda x: int(x))
16     return "".join(sorted(nums[:3], key=functools.cmp_to_key(cmp)))
17
18
19 # 算法调用
20 print(solution())
```

C算法源码

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4 #include <math.h>
5
6 #define MAX_LEN 2000
7 #define MAX_SIZE 200
8 #define MAX_NUM_LEN 10
9
```

```

10 int cmp(const void *a, const void *b) {11 |      char *A = (char *) a;
12     char *B = (char *) b;
13
14     return atoi(A) - atoi(B);
15 }
16
17 int cmp2(const void *a, const void *b) {
18     char *A = (char *) a;
19     char *B = (char *) b;
20
21     char s1[MAX_NUM_LEN * 2] = {'\0'};
22     strcat(s1, A);
23     strcat(s1, B);
24
25     char s2[MAX_NUM_LEN * 2] = {'\0'};
26     strcat(s2, B);
27     strcat(s2, A);
28
29     return strcmp(s1, s2);
30 }
31
32 int main() {
33     char s[MAX_LEN];
34     scanf("%s", s);
35
36     char nums[MAX_SIZE][MAX_NUM_LEN] = {'\0'};
37     int nums_size = 0;
38
39     char *token = strtok(s, ",");
40     while (token != NULL) {
41         strcpy(nums[nums_size++], token);
42         token = strtok(NULL, ",");
43     }
44
45     qsort(nums, nums_size, sizeof(nums[0]), cmp);
46
47     nums_size = (int) fmin(nums_size, 3);

```

```

48 |     qsort(nums, nums_size, sizeof(nums[0]), cmp2);
49 |
50 |     for (int i = 0; i < nums_size; i++) {
51 |         printf("%s", nums[i]);
52 |     }
53 |
54 |     return 0;
55 | }

```

C++ 算法源码

```

1 | #include <bits/stdc++.h>
2 |
3 | using namespace std;
4 |
5 | int main() {
6 |     string s;
7 |     cin >> s;
8 |
9 |     stringstream ss(s);
10 |    string token;
11 |
12 |    vector<string> nums;
13 |    while (getline(ss, token, ',')) {
14 |        nums.emplace_back(token);
15 |    }
16 |
17 |    sort(nums.begin(), nums.end(), [](string &a, string &b) {
18 |        return stoi(a) < stoi(b);
19 |    });
20 |
21 |    if (nums.size() > 3) {
22 |        nums.resize(3);
23 |    }
24 |

```

```
25 |     sort(nums.begin(), nums.end(), [](string &a, string &b) {  
26 |  
27 |     });  
28 |  
29 |     for (string &num: nums) {  
30 |         cout << num;  
31 |     }  
32 |  
33 |     return 0;  
34 | }
```

```
return (a + b) <= (b + a);
```